

Arquitectura de computadoras

Totalizador

Fecha: 02/08/2021

Alumno: D'ALU DE BONI, LISANDRO

Finalizado: No

Resultado:

Sala: <https://meet.jit.si/lisandrodalugmail.com>

Teoría

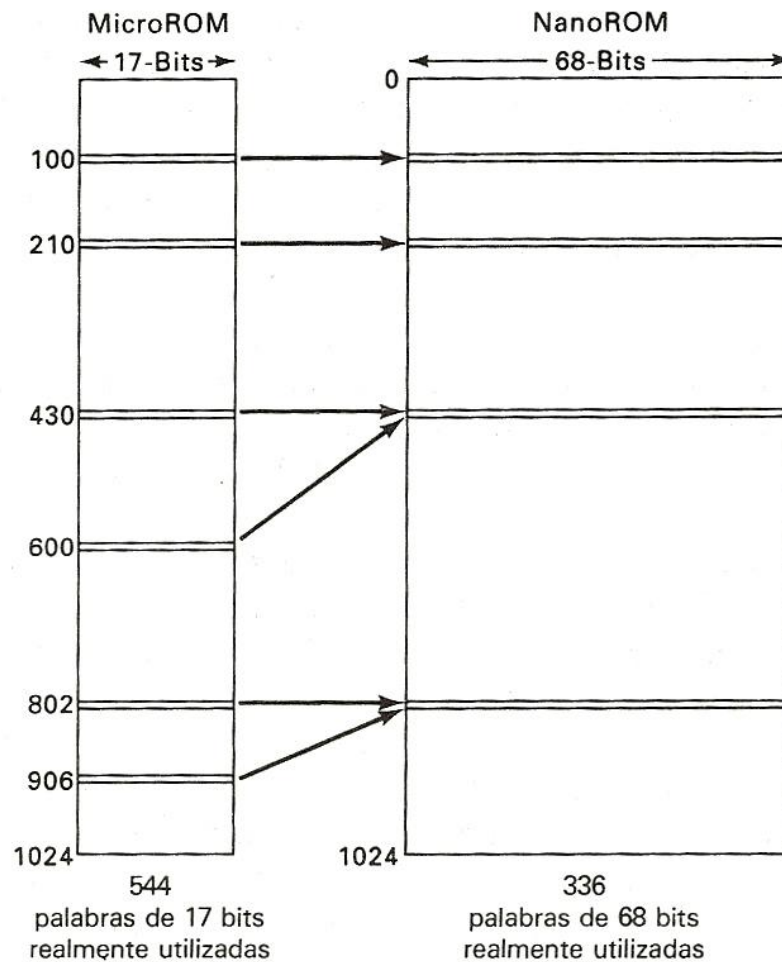


Fig. 4-35. El 68000 tiene un almacén de control de dos niveles con una microROM y una nanoROM.

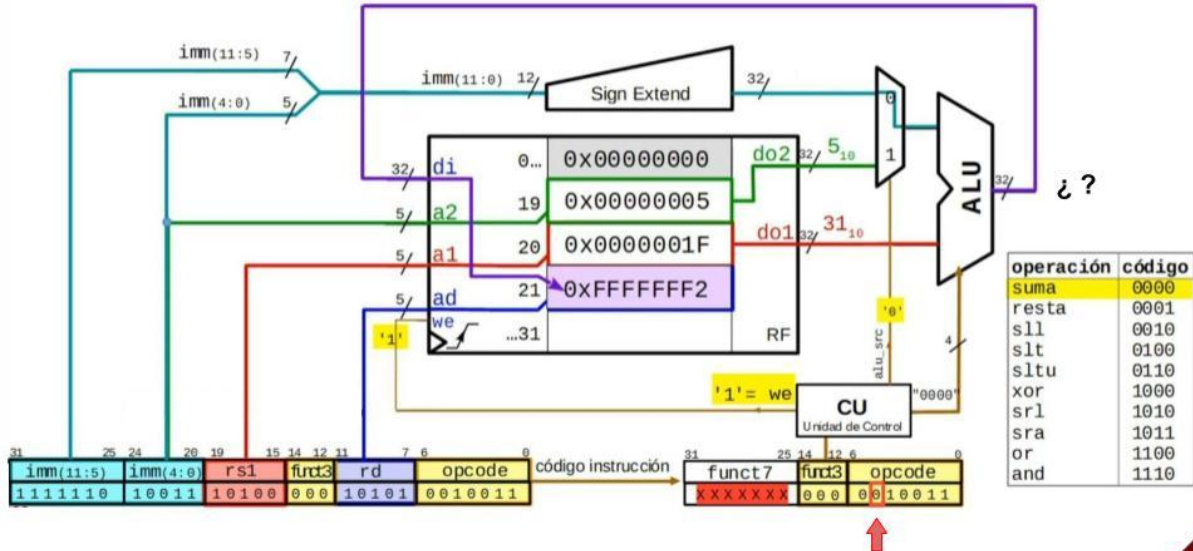
Principio de Localidad

100 CALL SUMAR

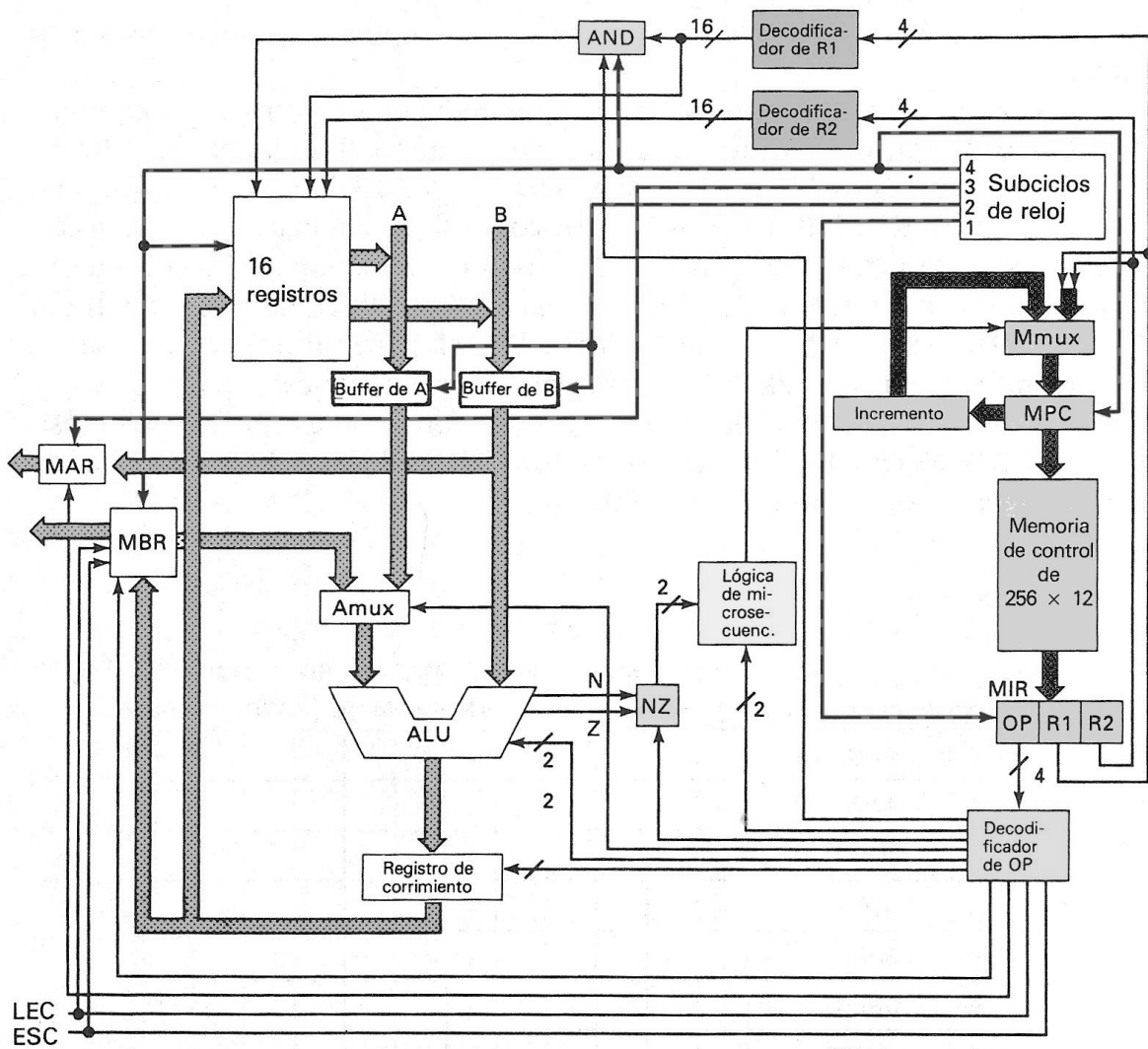
101 ADD SP,2

Describe el funcionamiento de la siguiente instrucción

`addi x21, x20, -45`



Código de operación de microinstrucción		ALUL		SHL		AMUX		MAR		RD		MSLH	
		ALUH	SHH			NZ	AND		MBR		WR	MSLL	
0	ADD					+	+						
1	AND		+			+	+						
2	MOVE	+				+	+						
3	COMPL	+	+			+	+						
4	LSHIFT	+		+		+	+						
5	RSHIFT	+			+	+	+						
6	GETMBR	+				+	+	+					
7	TEST	+				+							
8	BEGRD	+						+		+			
9	BEGWR	+						+	+		+		
10	CONRD	+								+			
11	CONWR	+									+		
12													
13	NJUMP	+											+
14	ZJUMP	+									+		
15	UJUMP	+									+	+	



Ejercicio Práctica

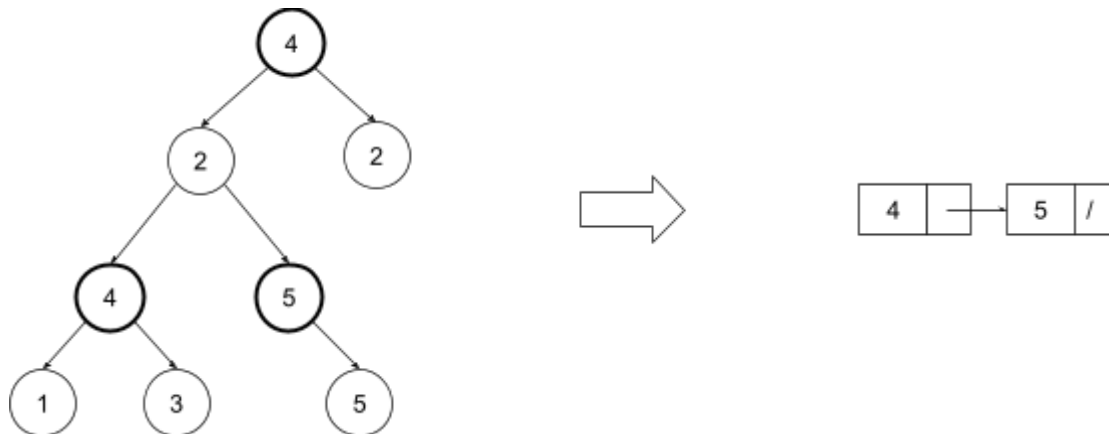
(Aquí se escribe el ejercicio sorteado)

Inicio 18:20

Tema 13

Crear una lista con los valores de todos los nodos de un árbol binario que cumplan con la condición que el valor es igual a la suma de los valores de sus hijos. Teniendo en cuenta que la lista debe quedar ordenada y no debe repetir valores.

Ejemplo:



La subrutina debe responder a la siguiente invocación:

Tlista *ObtenerLista (Tnodobinario *root);

Teniendo en cuenta la siguientes estructuras:



Consideraciones:

- Si utiliza una estructura auxiliar debe estar definida.
- En caso de que la subrutina utilice a otras subrutinas deben estar todas desarrolladas.
- Debe especificar todas las subrutinas.

Solución

(Aquí escribe la solución el alumno)

;Especificación de las todas las subrutinas a utilizar:

```
NULL EQU -1
izq EQU 1
der EQU 2
```

```
PUSH raiz
CALL OBTENERLISTA
ADD SP,1
```

;Subrutina que devuelve en AX NULL o el puntero al primer nodo de la lista
;[BP+2] = puntero al nodo raiz del arbol binario

```
OBTENERLISTA: PUSH BP
               MOV SP,BP
               PUSH CX
               PUSH DX

               MOV CX,1
               SYS 5
               MOV [DX],NULL; celda que contiene el primer nodo
```

```
               PUSH DX
               PUSH [BP+2]
               CALL ARMA_LISTA
               ADD SP,2
```

```
MOV AX,[DX]
```

```
FIN_OBTENER_LISTA: POP DX
                   POP CX
                   MOV SP,BP
                   POP BP
                   RET
```

;Subrutina que recorre recursivamente un arbol binario y verifica si el valor del nodo entra en la lista

;[BP+3] = puntero a primer nodo de la lista

;[BP+2] = puntero a nodo raiz del arbol binario

```
ARMA_LISTA: PUSH BP
            MOV SP,BP
            PUSH AX
```

```
PUSH BX
PUSH CX
```

```
MOV AX,[BP+2]; celda B
```

```
CMP AX,NULL
JZ FIN_ARMA_LISTA
```

```
MOV BX,0; lo uso para hacer la suma de los hijos
```

```
MOV CX,[BX+izq]
CMP [CX],NULL
JZ derecha
```

```
ADD BX,[CX]; le sumo el valor
```

```
derecha:  MOV CX,[BX+der]
          CMP [CX],NULL
          JZ comparo
```

```
          ADD BX,[CX]; le sumo el valor
comparo:  CMP [AX],BX
          JNZ nuevo
;Para insertar en la lista
```

```
PUSH [AX] ; valor a introducir
PUSH [BP+3]
CALL AGREGA_NODO
ADD SP,2
```

```
nuevo:    PUSH [BP+3]
          PUSH [BX+izq]
          CALL ARMA_LISTA
          ADD SP,2
```

```
PUSH [BP+3]
PUSH [BX+der]
CALL ARMA_LISTA
ADD SP,2
```

```
FIN_ARMA_LISTA: POP CX
                POP BX
                POP AX
                MOV SP,BP
                POP BP
                RET
```

```
;Subrutina que agrega un nodo dentro de la lista
;[BP+3] = valor
```

;[BP+2] = celda que contiene puntero a primer nodo

AGREGA_NODO: PUSH BP

MOV BP,SP

PUSH AX

PUSH BX

PUSH CX

PUSH DX

;creo el nodo

MOV CX,2

SYS 5

MOV [DX],[BP+3]

MOV [DX+1],NULL

MOV AX,[BP+2]; puntero a primer nodo de la lista

CMP [AX],NULL

JZ cabecera

MOV AX,[AX]; me muevo al nodo

MOV BX,NULL; lo utilizo como nodo anterior para enlazar

busco: CMP AX,NULL

JZ inserto

CMP [AX],[BP+3]; si es un valor que ya existe

JZ existe

JP inserto

MOV BX,AX; ant = act

MOV AX,[AX+1]; act = sig

JMP busco

inserto: MOV [DX+1],AX ; puede ser null o un nodo

CMP BX,NULL ; ant == NULL

JZ cabecera

MOV [BX+1],DX; ant->sig = nuevo

JMP FIN_AGREGA_NODO

cabecera: MOV [AX],DX

JMP FIN_AGREGA_NODO

existe: SYS 6; libero el nodo creado previamente

FIN_AGREGA_NODO: POP DX

POP CX

POP BX

POP AX

MOV SP,BP

POP BP

RET

